REPORT

Reviewer report: pytoolz/toolz#662: Add immutable

nested dictionary deletion with dissoc_in

Run toolz-662, tested 2026-11-11 12:39 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-11 12:43 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

NEEDS ATTENTION

Recommended action: Reject Confidence: Medium

The run is trustworthy and found a difference a person should look at. Some checks gave warnings: Probes ran

on both sides; Every claim cites a real log line

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Add immutable nested dictionary deletion with dissoc_in (pytoolz/toolz#662).

• What we observed: probe: 1 observation(s) need attention: probe.error differs and no claim covers it; probe:

claim(s) C2, C3, C4 proven; probe model-probe-c1 is not_adequate (the probe crashed or printed no parsed

output on every main-side run; not counted as ran or adequate); its matching results do not count; tests: 100

test(s) ran on main, 112 on the PR; 12 new test(s) pass, 101 unchanged (1 of them skipped on both sides).

• What you should do: Do not approve as is. Read 'What changed in behavior' and decide with the PR author.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 9.3 GB (of 15.6 GB).

• Setup check: free disk 92.1 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: toolz python setup install instructions https://github.com/pytoolz/toolz site:github.com/pytoolz/toolz

• Query: toolz python documentation install https://github.com/pytoolz/toolz site:github.com/pytoolz/toolz

• Query: toolz python install requirements https://pypi.org/project/toolz/ https://github.com/pytoolz/toolz

site:pypi.org

• Source: toolz/pyproject.toml at master · pytoolz/toolz · GitHub —

http://github.com/pytoolz/toolz/blob/master/pyproject.toml

• Source: toolz/pyproject.toml at master · pytoolz/toolz · GitHub —

https://github.com/pytoolz/toolz/blob/master/pyproject.toml

• Source: toolz/README.rst at master · pytoolz/toolz · GitHub —

https://github.com/pytoolz/toolz/blob/master/README.rst

• Source: toolz/README.rst at master · pytoolz/toolz · GitHub —

http://github.com/pytoolz/toolz/blob/master/README.rst

• Source: toolz - PyPI — https://pypi.org/project/toolz/

• Source: toolz/.github/workflows/test.yml at 451af60dec590a6010e2babdbf391ea8f815122f · pytoolz/toolz ·

GitHub —

https://github.com/pytoolz/toolz/blob/451af60dec590a6010e2babdbf391ea8f815122f/.github/workflows/test.yml

• Source: GitHub - pytoolz/toolz: A functional standard library for Python. · GitHub —

https://github.com/pytoolz/toolz

• Setup plan: the same packages on main and pr.

• Setup plan: install coverage (listed in the test extra) on main and pr.

• Setup plan: install pycodestyle (listed in the test extra) on main and pr.

• Setup plan: install pytest (listed in the test extra) on main and pr.

• Setup plan: install sphinx (listed in requirements.txt) on main and pr.

• Setup plan: install furo (listed in requirements.txt) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.9

• Setup plan: refused pip: its PyPI page does not link to this GitHub repository

• Setup plan: refused test: no network or the package was not found

• Installed coverage 7.17.2 from pyproject (listed in the test extra) on main and pr.

• Installed pycodestyle 2.15.0 from pyproject (listed in the test extra) on main and pr.

• Installed pytest 9.2.0 from pyproject (listed in the test extra) on main and pr.

• Installed sphinx 9.1.1 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed furo 2025.12.19 from requirements.txt (listed in requirements.txt) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (101 unchanged, 3 runs per side) | pytest-pr-1.log:8 «112 passed, 1 skipped in

0.94s»

| P-C1 | The function dissoc_in(d, keys, factory=dict) provides immutable nested dictionary deletion. (PR

description) | proven | probe model-probe-c1, c1.present: value: "<absent>" -> true |

probe-model-probe-c1-pr-1.jsonl:1 «"observation": "c1.present"»

| - | Change not in the PR description (probe model-probe-c1) | unmentioned change | probe.error: value:

{"crash": "non-json output", "exit_co... -> "<absent>" | no matching log line

| P-C2 | dissoc_in copies mappings along the selected path while preserving unrelated values by identity. (PR

description) | proven | probe model-probe-c2, c2.exception: value: "ImportError" -> "<absent>" | no matching log

line

| P-C3 | dissoc_in ignores missing keys in the path without raising an error. (PR description) | proven | probe

model-probe-c3, c3.exception: value: "ImportError" -> "<absent>" | no matching log line

| P-C4 | dissoc_in retains empty parent mappings after deletion. (PR description) | proven | probe

model-probe-c4, c4.exception: value: "ImportError" -> "<absent>" | no matching log line

Citation check (WARN): 2 citation(s) verified; 4 statement(s) have no matching log line.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 100 / 112 | | Tests unchanged | 101 | | New tests that pass | 12 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests

did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe-c1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c1.py, frozen

57290ecd691c at 2026-11-11T07:11:37.196440+00:00): Check existence of dissoc_in in top-level toolz module.

• P-C1 (PR description): The function dissoc_in(d, keys, factory=dict) provides immutable nested dictionary

deletion. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.present | P-C1 | as claimed | value: "<absent>" -> true | no matching log

line
probe-model-probe-c1-pr-1.jsonl:1 «"observation": "c1.present"»

| probe.error | - | differs, not in the PR description | value: {"crash": "non-json output", "exit_co... -> "<absent>" |

probe-model-probe-c1-main-1.jsonl:1 «"observation": "probe.error"»
no matching log line

model-probe-c2 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c2.py, frozen

dc6ae5ec7e04 at 2026-11-11T07:11:37.196593+00:00): Attempt to call dissoc_in to see if it copies mappings

and preserves identity (will fail if missing).

• P-C2 (PR description): dissoc_in copies mappings along the selected path while preserving unrelated values by

identity. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c2.exception | P-C2 | as claimed | value: "ImportError" -> "<absent>" | probe-model-probe-c2-main-1.jsonl:1

«"observation": "c2.exception"»
no matching log line

| c2.result | P-C2 | as claimed | value: "<absent>" -> {"a": {}} | no matching log

line
probe-model-probe-c2-pr-1.jsonl:1 «"observation": "c2.result"»

model-probe-c3 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c3.py, frozen

c1e4c279ee43 at 2026-11-11T07:11:37.196676+00:00): Attempt to call dissoc_in with missing key in path to see

if it ignores missing keys (will fail if missing).

• P-C3 (PR description): dissoc_in ignores missing keys in the path without raising an error. Expects the covered

observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c3.exception | P-C3 | as claimed | value: "ImportError" -> "<absent>" | probe-model-probe-c3-main-1.jsonl:1

«"observation": "c3.exception"»
no matching log line

| c3.result | P-C3 | as claimed | value: "<absent>" -> {"a": {"b": 1}} | no matching log

line
probe-model-probe-c3-pr-1.jsonl:1 «"observation": "c3.result"»

model-probe-c4 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c4.py, frozen

ee9e5807a707 at 2026-11-11T07:11:37.196755+00:00): Attempt to call dissoc_in deleting an empty mapping to

see if empty parent mappings are retained (will fail if missing).

• P-C4 (PR description): dissoc_in retains empty parent mappings after deletion. Expects the covered

observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c4.exception | P-C4 | as claimed | value: "ImportError" -> "<absent>" | probe-model-probe-c4-main-1.jsonl:1

«"observation": "c4.exception"»
no matching log line

| c4.result | P-C4 | as claimed | value: "<absent>" -> {"a": {}} | no matching log

line
probe-model-probe-c4-pr-1.jsonl:1 «"observation": "c4.result"»

diff-api (diff (auto-derived from base-side code, model-free); informational under baseline full,

diff-probes/diff-api.py, frozen 44bf4a0a3bdf at 2026-11-11T07:11:37.198558+00:00): Public functions touched by

the PR, called with type-appropriate inputs on both sides.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| toolz.dicttoolz.assoc_in.input0 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:1 «"observation": "toolz.dicttoolz.assoc_in.input0"»
probe-diff-api-pr-1.jsonl:1

«"observation": "toolz.dicttoolz.assoc_in.input0"»

| toolz.dicttoolz.assoc_in.input1 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:2 «"observation": "toolz.dicttoolz.assoc_in.input1"»
probe-diff-api-pr-1.jsonl:2

«"observation": "toolz.dicttoolz.assoc_in.input1"»

| toolz.dicttoolz.assoc_in.input10 | - | probe could not run | same on both sides: {"raises": "StopIteration"} |

probe-diff-api-main-1.jsonl:3 «"observation": "toolz.dicttoolz.assoc_in.input10"»
probe-diff-api-pr-1.jsonl:3

«"observation": "toolz.dicttoolz.assoc_in.input10"»

| toolz.dicttoolz.assoc_in.input11 | - | probe could not run | same on both sides: {"raises": "StopIteration"} |

probe-diff-api-main-1.jsonl:4 «"observation": "toolz.dicttoolz.assoc_in.input11"»
probe-diff-api-pr-1.jsonl:4

«"observation": "toolz.dicttoolz.assoc_in.input11"»

| toolz.dicttoolz.assoc_in.input2 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:5 «"observation": "toolz.dicttoolz.assoc_in.input2"»
probe-diff-api-pr-1.jsonl:5

«"observation": "toolz.dicttoolz.assoc_in.input2"»

| toolz.dicttoolz.assoc_in.input3 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:6 «"observation": "toolz.dicttoolz.assoc_in.input3"»
probe-diff-api-pr-1.jsonl:6

«"observation": "toolz.dicttoolz.assoc_in.input3"»

| toolz.dicttoolz.assoc_in.input4 | - | probe could not run | same on both sides: {"raises": "ValueError"} |

probe-diff-api-main-1.jsonl:7 «"observation": "toolz.dicttoolz.assoc_in.input4"»
probe-diff-api-pr-1.jsonl:7

«"observation": "toolz.dicttoolz.assoc_in.input4"»

| toolz.dicttoolz.assoc_in.input5 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:8 «"observation": "toolz.dicttoolz.assoc_in.input5"»
probe-diff-api-pr-1.jsonl:8

«"observation": "toolz.dicttoolz.assoc_in.input5"»

| toolz.dicttoolz.assoc_in.input6 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:9 «"observation": "toolz.dicttoolz.assoc_in.input6"»
probe-diff-api-pr-1.jsonl:9

«"observation": "toolz.dicttoolz.assoc_in.input6"»

| toolz.dicttoolz.assoc_in.input7 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:10 «"observation": "toolz.dicttoolz.assoc_in.input7"»
probe-diff-api-pr-1.jsonl:10

«"observation": "toolz.dicttoolz.assoc_in.input7"»

| toolz.dicttoolz.assoc_in.input8 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:11 «"observation": "toolz.dicttoolz.assoc_in.input8"»
probe-diff-api-pr-1.jsonl:11

«"observation": "toolz.dicttoolz.assoc_in.input8"»

| toolz.dicttoolz.assoc_in.input9 | - | probe could not run | same on both sides: {"raises": "StopIteration"} |

probe-diff-api-main-1.jsonl:12 «"observation": "toolz.dicttoolz.assoc_in.input9"»
probe-diff-api-pr-1.jsonl:12

«"observation": "toolz.dicttoolz.assoc_in.input9"»

| toolz.dicttoolz.dissoc.input0 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:13 «"observation": "toolz.dicttoolz.dissoc.input0"»
probe-diff-api-pr-1.jsonl:13

«"observation": "toolz.dicttoolz.dissoc.input0"»

| toolz.dicttoolz.dissoc.input1 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:14 «"observation": "toolz.dicttoolz.dissoc.input1"»
probe-diff-api-pr-1.jsonl:14

«"observation": "toolz.dicttoolz.dissoc.input1"»

| toolz.dicttoolz.dissoc.input10 | - | same on both sides | same on both sides: {} | probe-diff-api-main-1.jsonl:15

«"observation": "toolz.dicttoolz.dissoc.input10"»
probe-diff-api-pr-1.jsonl:15 «"observation":

"toolz.dicttoolz.dissoc.input10"»

| toolz.dicttoolz.dissoc.input11 | - | same on both sides | same on both sides: {} | probe-diff-api-main-1.jsonl:16

«"observation": "toolz.dicttoolz.dissoc.input11"»
probe-diff-api-pr-1.jsonl:16 «"observation":

"toolz.dicttoolz.dissoc.input11"»

| toolz.dicttoolz.dissoc.input2 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:17 «"observation": "toolz.dicttoolz.dissoc.input2"»
probe-diff-api-pr-1.jsonl:17

«"observation": "toolz.dicttoolz.dissoc.input2"»

| toolz.dicttoolz.dissoc.input3 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:18 «"observation": "toolz.dicttoolz.dissoc.input3"»
probe-diff-api-pr-1.jsonl:18

«"observation": "toolz.dicttoolz.dissoc.input3"»

| toolz.dicttoolz.dissoc.input4 | - | probe could not run | same on both sides: {"raises": "ValueError"} |

probe-diff-api-main-1.jsonl:19 «"observation": "toolz.dicttoolz.dissoc.input4"»
probe-diff-api-pr-1.jsonl:19

«"observation": "toolz.dicttoolz.dissoc.input4"»

| toolz.dicttoolz.dissoc.input5 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:20 «"observation": "toolz.dicttoolz.dissoc.input5"»
probe-diff-api-pr-1.jsonl:20

«"observation": "toolz.dicttoolz.dissoc.input5"»

| toolz.dicttoolz.dissoc.input6 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:21 «"observation": "toolz.dicttoolz.dissoc.input6"»
probe-diff-api-pr-1.jsonl:21

«"observation": "toolz.dicttoolz.dissoc.input6"»

| toolz.dicttoolz.dissoc.input7 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:22 «"observation": "toolz.dicttoolz.dissoc.input7"»
probe-diff-api-pr-1.jsonl:22

«"observation": "toolz.dicttoolz.dissoc.input7"»

| toolz.dicttoolz.dissoc.input8 | - | probe could not run | same on both sides: {"raises": "TypeError"} |

probe-diff-api-main-1.jsonl:23 «"observation": "toolz.dicttoolz.dissoc.input8"»
probe-diff-api-pr-1.jsonl:23

«"observation": "toolz.dicttoolz.dissoc.input8"»

| toolz.dicttoolz.dissoc.input9 | - | same on both sides | same on both sides: {} |
probe-diff-api-main-1.jsonl:24

«"observation": "toolz.dicttoolz.dissoc.input9"»
probe-diff-api-pr-1.jsonl:24 «"observation":

"toolz.dicttoolz.dissoc.input9"»

Citation check (PASS): 56 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (2.5 s run).

Coverage gate (python, confidence normal): 24 of 24 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| toolz/curried/__init__.py | 1 | 1

| toolz/dicttoolz.py | 23 | 23

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested a033e5da039d of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 24 of 24 changed executable line(s) ran (100%).

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every probe observation cites its output line: 56 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Probes ran on both sides | WARN | model-probe-c1 main#1; model-probe-c1 main#2; model-probe-c1 main#3 |

Read the probe-*.log files; a probe that cannot run is not evidence either way.

| Every claim cites a real log line | WARN | 2 citation(s) verified; 4 statement(s) have no matching log line |

Regenerate the report; if it persists, the logs were edited or truncated.

| Behavior difference found | finding | probe: 1 observation(s) need attention: probe.error differs and no claim

covers it; probe: claim(s) C2, C3, C4 proven; probe model-probe-c1 is not_adequate (the probe crashed or

printed no parsed output on every main-side run; not counted as ran or adequate); its matching results do not

count; tests: 100 test(s) ran on main, 112 on the PR; 12 new test(s) pass, 101 unchanged (1 of them skipped on

both sides) | This is about the PR, not the run: read the failing tests and their log lines, then decide.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 24 of 24 changed executable line(s) ran (100%)

| Probes ran on both sides | WARN | model-probe-c1 main#1; model-probe-c1 main#2; model-probe-c1 main#3

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | WARN | 2 citation(s) verified; 4 statement(s) have no matching log line

| Every probe observation cites its output line | PASS | 56 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-11/toolz-662 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/pytoolz/toolz/pull/662

Evidence links

• Pull request pytoolz/toolz#662

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-probe-c1-main-1.jsonl

• probe-model-probe-c1-pr-1.jsonl

• probe-model-probe-c2-main-1.jsonl

• probe-model-probe-c2-pr-1.jsonl

• probe-model-probe-c3-main-1.jsonl

• probe-model-probe-c3-pr-1.jsonl

• probe-model-probe-c4-main-1.jsonl

• probe-model-probe-c4-pr-1.jsonl

• probe-diff-api-main-1.jsonl

• probe-diff-api-pr-1.jsonl

• main run 1 result: pytest-main-1.log:8 «100 passed, 1 skipped in 0.98s»

• pr run 1 result: pytest-pr-1.log:8 «112 passed, 1 skipped in 0.94s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-11 12:44 IST